

Two sum:

1. Brute Force

The screenshot shows the NeetCode interface for the 'Two Sum' problem. The submission is marked as 'Accepted' for 23/23 test cases. The performance metrics are: Runtime 27ms (Beats 100.00%) and Memory 7.7 MB (Beats 99.98%). A bar chart shows the runtime distribution with a peak at 28ms. The code is a Python class solution using a nested loop to find two numbers that sum to the target.

Runtime: 27ms | Beats 100.00%
Memory: 7.7 MB | Beats 99.98%

Passed test cases: 23 / 23

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        for i in range(len(nums)):
            for j in range(i+1, len(nums)):
                if nums[i] + nums[j] == target:
                    return [i, j]
```

2. Hash map

The screenshot shows the NeetCode interface for the 'Two Sum' problem. The submission is marked as 'Accepted' for 23/23 test cases. The performance metrics are: Runtime 36ms (Beats 27.19%) and Memory 8.0 MB (Beats 21.77%). A bar chart shows the runtime distribution with a peak at 28ms. The code is a Python class solution using a hash map to store the complement of each number.

Runtime: 36ms | Beats 27.19%
Memory: 8.0 MB | Beats 21.77%

Passed test cases: 23 / 23

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        seen = {}
        for i in range(len(nums)):
            current_num = nums[i]
            complement = target - current_num
            if complement in seen:
                return [seen[complement], i]
            seen[current_num] = i
```

Two Sum — Complexity Comparison

Method	Idea	Time Complexity	Space Complexity	Why?
Brute Force	Check every pair	$O(n^2)$	$O(1)$	Nested loops
Hash Map	Store visited numbers in dictionary	$O(n)$	$O(n)$	Fast dictionary lookup

SUPER SHORT INTERVIEW SUMMARY

Python

Brute Force:

- checks all pairs
- $O(n^2)$ time
- $O(1)$ space

Hash Map:

- uses complement logic
- $O(n)$ time
- $O(n)$ space
- faster due to $O(1)$ lookup